CS 391
Web Application Development

Esma Meral

esma.meral@ozyegin.edu.tr

Week - 6

Outline

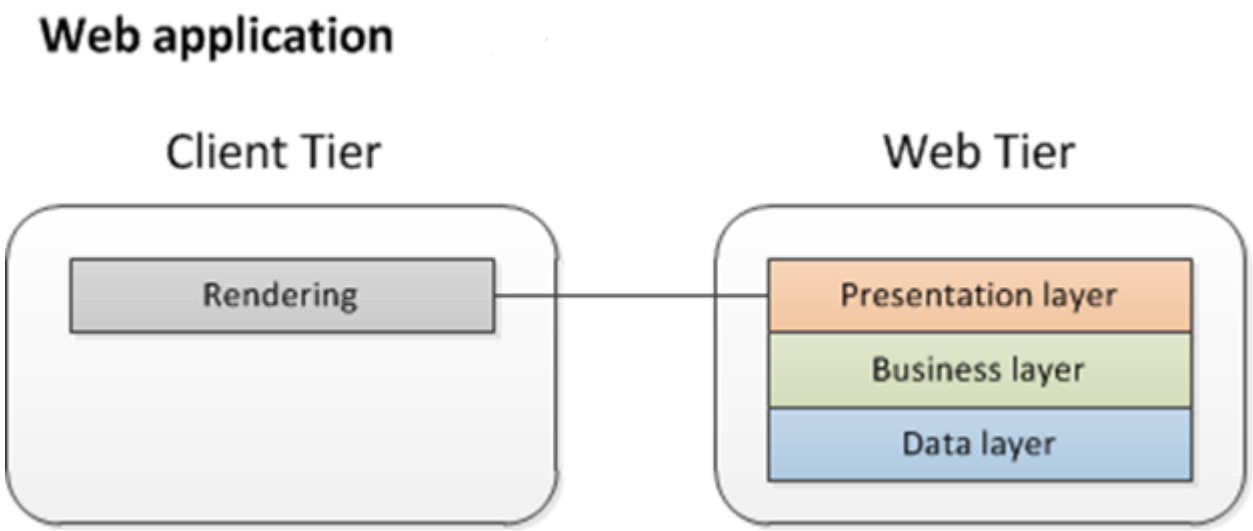
1. Create Dynamic Web Pages : JavaScript Frameworks
2. JavaScript modules
3. What is NodeJS?
4. Node Package Manager (NPM)

Create Dynamic Web Pages : JavaScript Frameworks

1. Create Dynamic Web Pages : JavaScript Frameworks
2. JavaScript modules
3. What is NodeJS?
4. Node Package Manager (NPM)

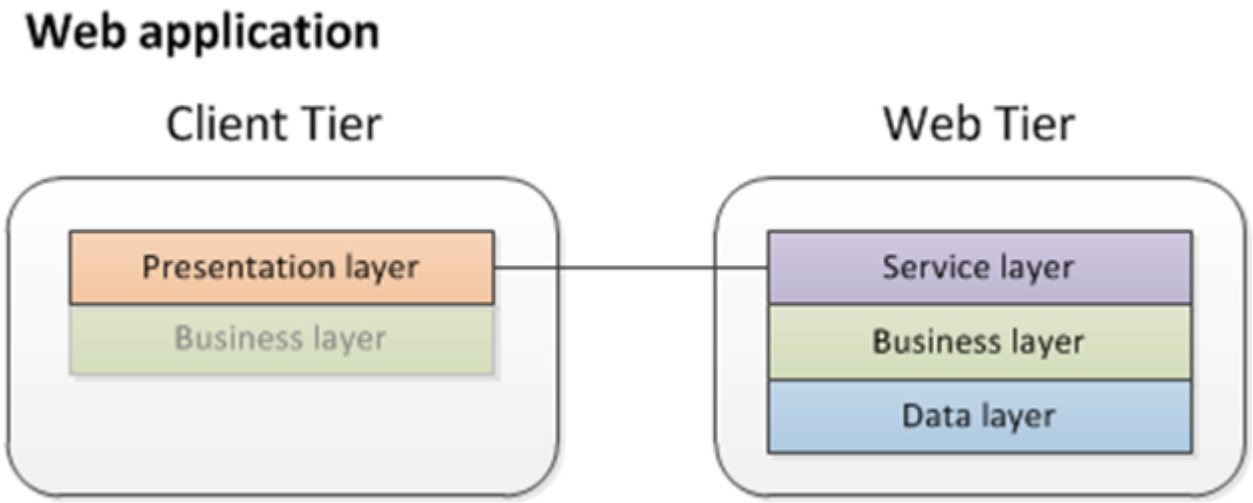
Presentation Layer

- In a “traditional” web application, presentation layer is within web tier, renders output to client tier (browser)



Presentation Layer

- Nowadays web sites are turning into Web apps
- Presentation layer moves to client tier
- May also have an element of business logic in client tier



Client Tier presentation

- Presentation layer technologies are HTML, JavaScript, CSS
- Easy to end up with spaghetti code – global variables, functions whose purpose is unclear, etc.
- Can improve this situation using patterns which have been tried and tested in the presentation layer in the Web Tier

What is MVC?

- Separates development between Data, Code and UI
- Each layer can be developed independently
- Minimizes turnaround time
- Less overlap between developers and designers
- Views can be swapped out for mobile, desktop, apps etc.

Most Popular Frameworks

- Most popular frameworks/libraries:
  - ReactJS => https://reactjs.org/
  - Angular.js => https://angular.io/
  - VueJs => https://vuejs.org/

TodoMVC

Helping you select an MV* framework

- http://todomvc.com/

todos

Response email

call John

~~Edit analysis document~~

1 item left | All Active Completed | Clear completed (1)

Examples

JavaScript | Compile-to-JS | Labs

These are examples written in pure JavaScript.

React New	React Redux New	Vue.js New
Preact New	Backbone.js New	Angular New
Ember.js New	Lit New	KnockoutJS
Dojo	Knockback.js	CanJS
Polymer	Mithril	Marionette.js

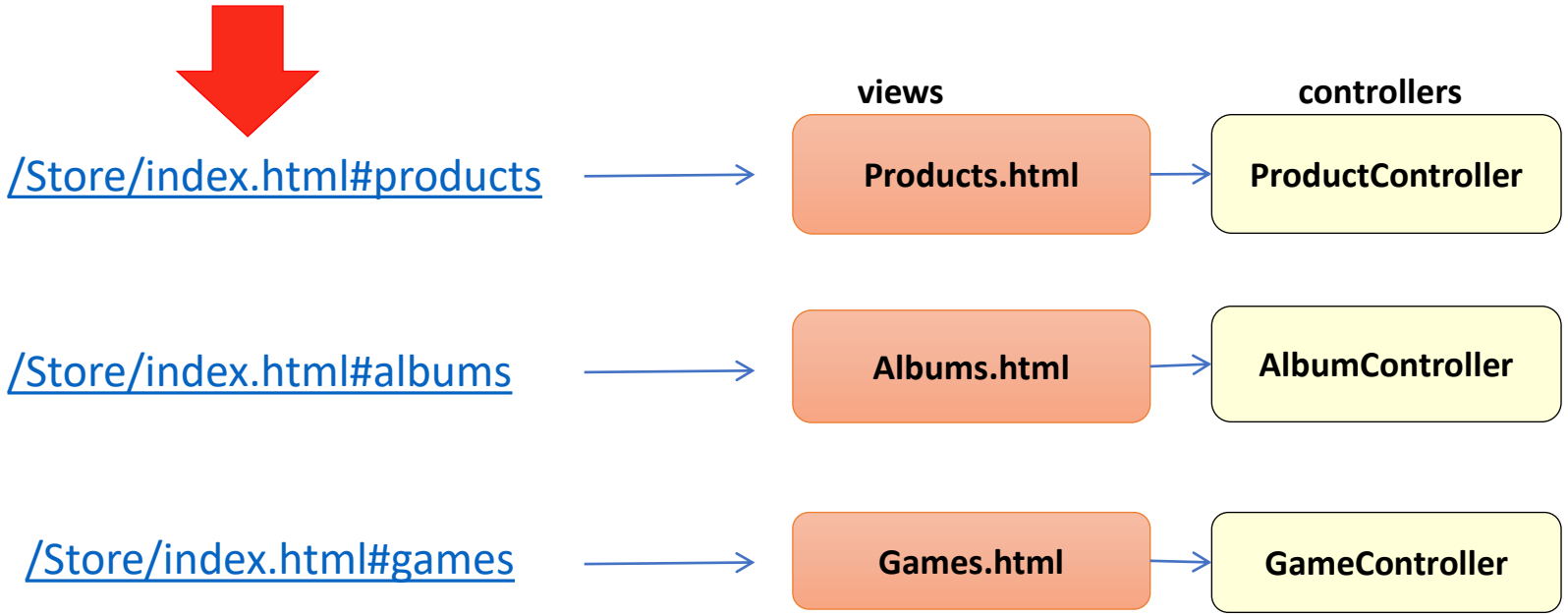
Single Page Application (SPA)

- Have a shell page load multiple views into that page
- Load the initial content upfront
  - then the different views or the little kind of mini- web pages can be loaded on the fly and embedded into the shell

...Single Page Application (SPA)

- A route is specified in the URL after the # sign

following URL's all point to the same html page, but each point to different routes



JavaScript modules

1. Create Dynamic Web Pages : JavaScript Frameworks
2. JavaScript modules
3. What is NodeJS?
4. Node Package Manager (NPM)

ES6 Modules…

- ES6 supports modules
  - A way to write JavaScript in different files
    - Each file has its own scope (not the global)
    - Each file decides what to export from its module
  - Export the objects you want from a module:

```
module.exports = {
...
  Person: Person,
  Mammal: Mammal
}
```

person.js

To use the module in another file:

```
import {Mammal, Person} from './person'
```

- Encapsulate behavior
- Easy to work with
- Easy to maintain
- Easy to scale

Exporting a Module

- The *export* keyword can be used to export components in a module.
- Exports in a module can be classified as follows:
  - Named Exports
  - Default Exports

Named Exports

- Named exports are distinguished by their names.
- Named exports are useful to export several values.
- There can be several named exports in a module.
- Two syntaxes:

```
//using multiple export keyword
function component1() { … }
function component2() { … }
. . .
export component1
export component2
…
…
export componentN
```

```
//using single export keyword
function component1() { … }
function component2() { … }

export {component1,component2,....,componentN}
```

Importing Named Exports

- During the import, it is mandatory to use the same name of the corresponding object.
- They are imported with curly brackets

```
// Named imports
import {  component1, component2 }   from './operations'
```

- However, while importing named exports, they can be renamed using the as keyword:

```
import {component1 as c1} from './operations'
```

You can only use import and export statements inside modules, not regular scripts.

Default Export

- Modules that need to export ***only a single*** value can use default exports.
- There can be only one default export per module

operations.js

```
function component1() { … }

export default component1
```

Importing and Using Default Exports

- Unlike named exports, a default export can be imported with any name.

```
import comp1 from ‘./operations.js'

console.log(comp1.component1())
```

If you try to load the HTML file locally (i.e. with a file:// URL), you'll run into CORS errors due to JavaScript module security requirements. You need to do your testing through a server

What is NodeJS?

1. Create Dynamic Web Pages : JavaScript Frameworks
2. JavaScript modules
3. What is NodeJS?
4. Node Package Manager (NPM)

What is NodeJS?

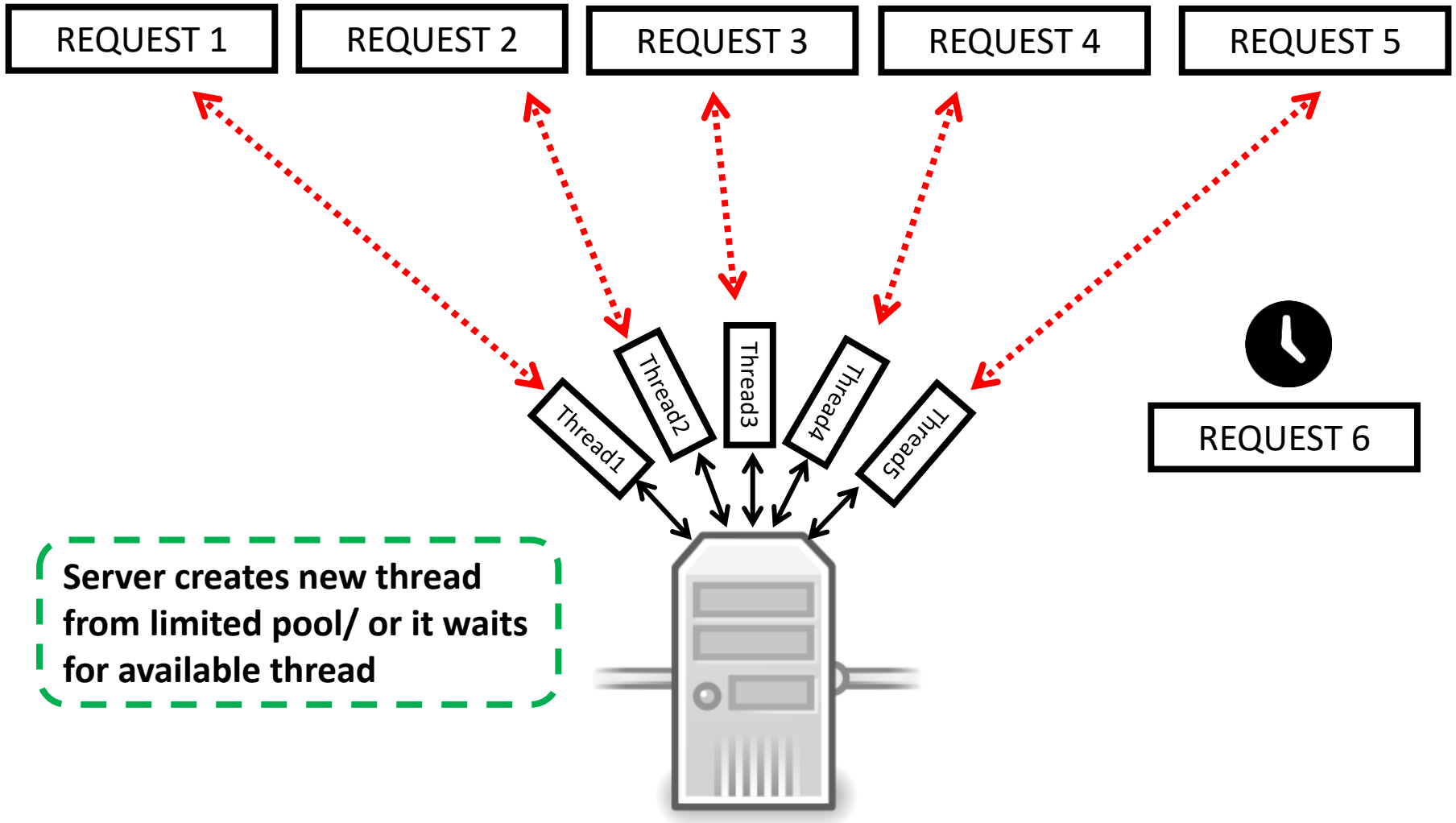
- **Node.js® :** Platform built on Chrome's JavaScript runtime for easily building, fast, scalable network applications.

https://nodejs.org/

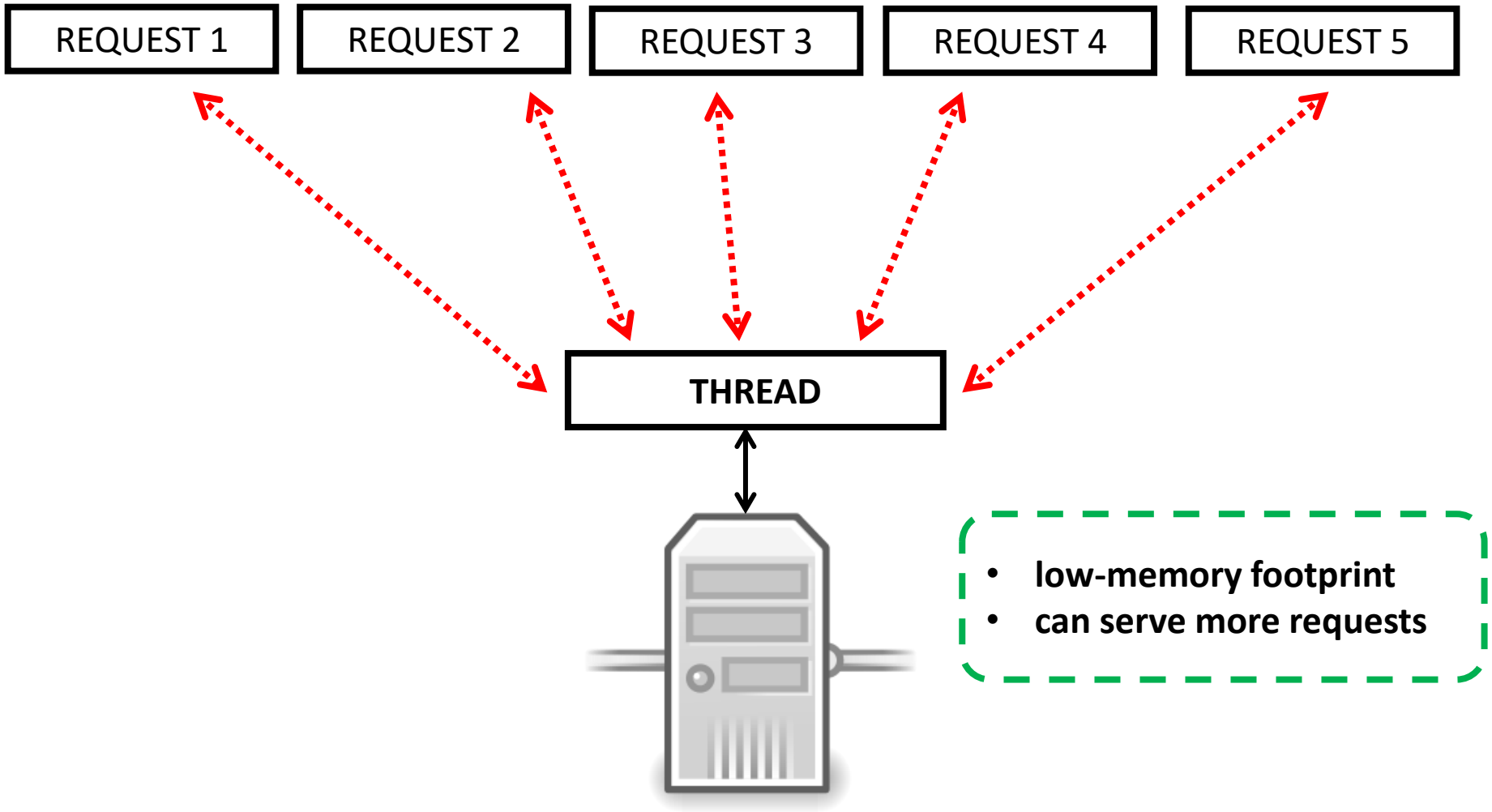
Features

- Set of libraries built on top of the Google's open source V8 JavaScript engine
- Server-side JavaScript environment
  - Node removes a layer of complexity by combining server and application logic in one place
- Event-driven
- Non-Blocking I/O
- Asynchronous
- Many more modules

Traditional systems



Node.js



- **low-memory footprint**
- **can serve more requests**

Understand non-blocking I/O

- A typical experience at a restaurant would be something like this:
  - You sit at a table, and the server grabs your drink order.
  - The server goes back to the bar and passes your order to a bartender.
  - While the bartender is working on your drink, the server moves on to grab another table's drink order.
  - The server goes back to the bar and passes along the other table's order.
  - Before the server brings back your drinks, you order some food.
  - Server passes your food order to the kitchen.
  - Your drinks are ready now, so the server picks them up and brings them back to your table.
  - The other table's drinks are ready, so the server picks them up and takes them to the other table.
  - Finally, your food is ready, so server picks it up and brings it back to your table.

So what is “non-blocking programming”?

- There are operations that take a long time to finish
  - File access
  - Network
  - DB operations
- What sync JavaScript code does is
  - Waits for results to complete
  - Does not handle other requests in the meantime.
- What node does is
  - Instead of waiting for the task to complete, it allows the processor to handle other requests in the meantime.
- Callbacks

Non-blocking vs. Blocking

```
// Blocking style

console.log("Accessing user info...")
var user = User.findOne({username: "john"});
console.log("User found: " + user);

console.log("Accessing user comments...")
var comment = Comment.findOne({username: "john"});
console.log("Comment found: " + comment);
```

Output:

Accessing user info...
User found: john
Accessing user comments...
Comment found: Node is cool!

Non-blocking vs. Blocking

```
// Non-blocking style
console.log("Accessing user info...")
User.findOne({username: "john"},
        function(err, userName) {
             console.log("User found: " + userName);}
);

console.log("Accessing user comments...")
Comment.findOne({username: "john"},
         function(err, comment) {
           console.log("Comment found: " + comment);}
);
```

Output:
Accessing user info...
Accessing user comments...
User found: john
Comment found: Node is cool!

Node Package Manager (NPM)

1. Create Dynamic Web Pages : JavaScript Frameworks
2. JavaScript modules
3. What is NodeJS?
4. Node Package Manager (NPM)

What is Npm?

- **NPM :** Node Package Manager.
  - a package manager for Node.js packages, or modules
- The registry contains many packages.
- Open-source developers use npm to share software.

Installation

- Go to https://nodejs.org/en/download and download nodejs installer file
- Execute downloaded file

Or get a prebuilt Node.js® for Windows running a x64 architecture.

Windows Installer (.msi) | Standalone Binary (.zip)

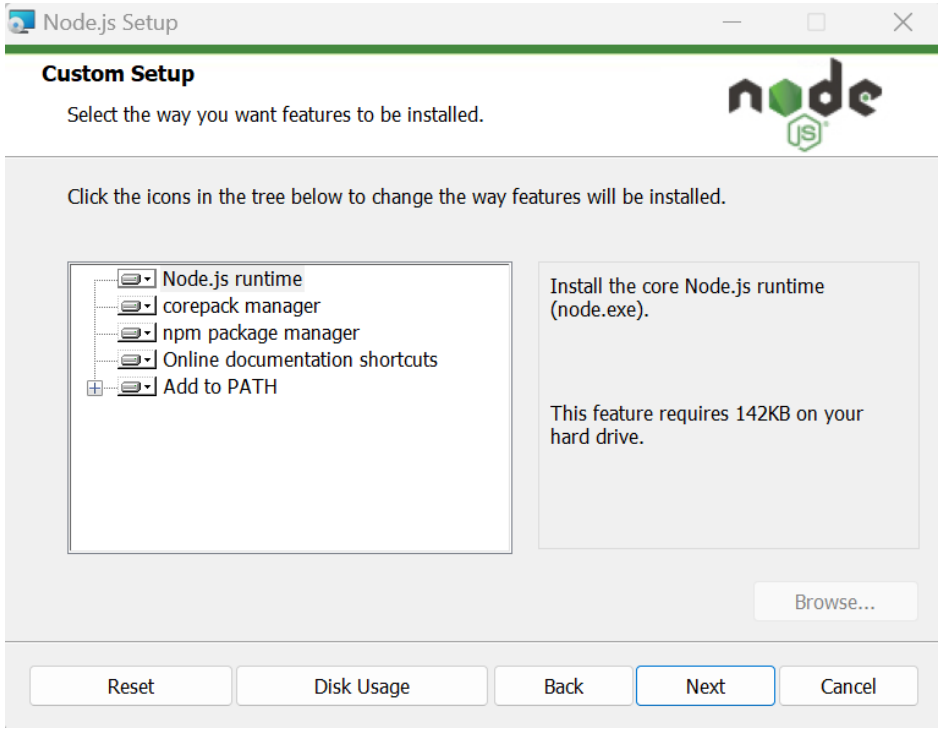
Read the changelog or blog post for this version.

Learn more about Node.js releases, including the release schedule and LTS status.

Learn how to verify signed SHASUMS.

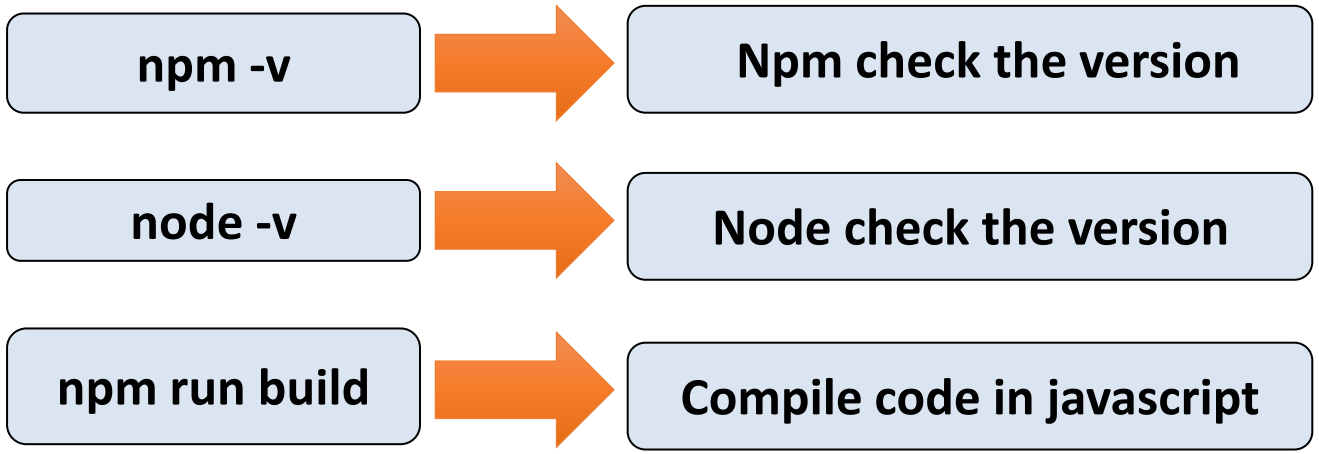
Looking for Node.js source? Download a signed Node.js source tarball.

Check out our nightly binaries or all previous releases or the unofficial binaries for other platforms.



Check Installation

- The NPM program is installed on your computer when you install Node.js



What is a Package?

- Packages are libraries that are included in the program
- These must be installed in every project using npm
  - Installing is normally as simple as downloading a few JavaScript source files

Software Package Manager

- All npm packages are defined in files called package.json
- The content of package.json must be written in JSON format.
- At least two fields must be present in the definition file: name and version.

```
{
    "name": "npmdemo",
    "version": "1.0.0",
    "description": "This is a npm package",
    "main": "index.js",
    "keywords": ["es6", "node", "react"],
    "author": "Esma Meral",
    "license": "MIT",
}
```

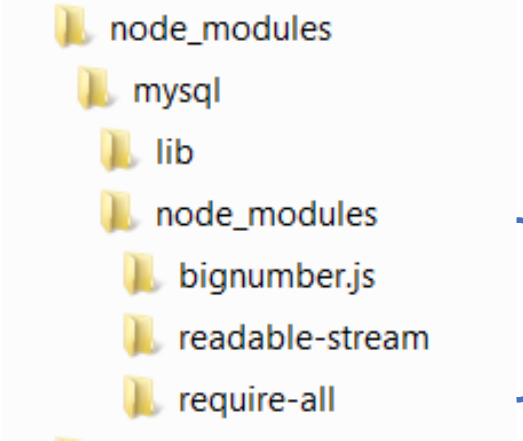
Installing Packages…

- Type `npm install <package-name>`
  - install the latest version
  - a version number also can be specified

```
D:\>npm install mysql
mysql@2.6.1 node_modules\mysql
├── require-all@1.0.0
├── bignumber.js@2.0.5
└── readable-stream@1.1.13 (string_decoder@0.10.31, isarray@0.0.1,
```

To specify version number:
> npm install mysql@2.6.0

installs in a subdirectory named **node_modules** within the current directory



dependencies

Installing Packages...

- npm install takes 3 exclusive, optional flags :
  - **--save:** Package will appear in the dependencies
    - unnecessary feature in the npm 5.0.0 version onwards
  - **--save-dev:** Package will appear in the devDependencies
  - **--save-optional:** Package will appear in the optionalDependencies
- For more commands:
  - https://docs.npmjs.com/cli/v11/commands

Installing Packages...

- Two modes:
  - Global mode
    - Install modules in {prefix}/lib/node_modules, where {prefix} is usually something like /usr/local
      - Use "npm config list" to see default {prefix} location
  - Local mode
    - npm installs the packages under current directory
    - packages goes under ./node_modules
  - Default mode is local mode

Global Packages…

- Global packages need to be installed in one location only
- They are not included in the code libraries
  - normally contain command line tools, which should be included in the PATH environment variable

...Global Packages

- To process a package globally, simply add `-g` or `--global` option to npm commands

```
// Installing package as global
npm install -g mysql
```

The *package.json* File

- Contains project's information
- Located in the project’s root directory

```
{
  "name": "myapp",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
  },
  "author": “Esma Meral",
  "license": "MIT",
  "dependencies": {
      "body-parser": "~1.12.0",
      "express": "^4.16.4",
  }
}
```

important attributes

- **Name**
- **Version**
- **Description**
- **Authors**
- **Contributors**
- **Main entry**
- **Dependencies**
- **Dev dependencies**
- **Engines**
- **Scripts**

Listing Installed Packages...

- npm ls
- npm ls -g // for global packages
  - Listing all of the currently installed packages

```
D:\dev\workspaces\nodejs-workspace\myapp>npm ls
myapp@0.0.0 D:\dev\workspaces\nodejs-workspace\myapp
├── body-parser@1.12.2
│   ├── bytes@1.0.0
│   ├── content-type@1.0.1
│   ├── depd@1.0.0
│   ├── iconv-lite@0.4.7
│   ├┬ on-finished@2.2.0
│   │└── ee-first@1.1.0
│   ├── qs@2.4.1
│   ├── raw-body@1.3.3
│   └┬ type-is@1.6.1
│    ├── media-typer@0.3.0
│    └┬ mime-types@2.0.10
│     └── mime-db@1.8.0
├── connect-flash@0.1.1 extraneous
```

Uninstalling npm packages

- For locally installed packages:
  - npm uninstall <package-name>
- For globally installed packages:
  - npm uninstall -g <package-name>

Searching Packages

```
// This command searches for packages in npmjs.com
npm search <library-name>
```

Or you can use web site: https://www.npmjs.com/

```
D:\>npm search knockout
NAME                                    DESCRIPTION
backbone-ko                             Knockout integration with Backbone.js
backstage                               backbone to knockout connector
baseviewmodel                           Backbone like model extension for knockout viewmodels,…
bindie                                  CommonJS/Browserified simple data view binding, without…
byrnie                                  Weaving Mithril into beautiful component armour
ccnq-ko                                 CCNQ Knockout Widget toolbox
ccnq-ko-rule-entry                      CCNQ Knockout Widget for rule entry
ccnq-ko-rule-gwlist                     CCNQ Knockout Widget for gwlist management
ccnq-ko-rule-target                     CCNQ Knockout Widget for rule target
ChartTime                               Hierarchical time-series axis for charts with knockouts for…
compute                                 A simple reactive programming library.
decojs                                  Scalable frontend architecture
doogle                                  Node.js app for taking HTML snapshots of JavaScript pages…
```

Create a Module

- The following steps show how you can use NPM to create a module
- Step 1 - Create a directory for new application

```
mkdir myapp
cd myapp
```

Create a package.json file

- Step 2- Use the **npm init** command to create a `package.json` file for application.

```
npm init
```

- Step 3- Install express in the myapp directory and save it in the dependency list of your `package.json` file

```
npm install express --save
```

unnecessary feature in the npm 5.0.0 version onwards

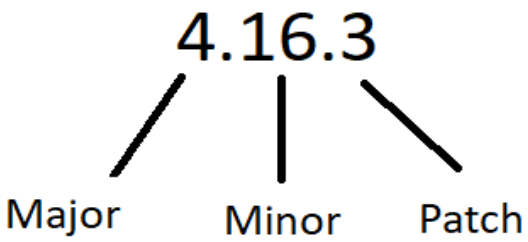
Check package.json file

- The dependencies section of your package.json will now appear at the end of the package.json file and will include Express.

```
// package.json
{
    "name": "myapp",
    "version": "1.0.0",
    "description": "",
    "main": "index.js",
    "scripts": {
        "test": "echo \"Error: no
test specified\" && exit 1",
    },
    "author": "",
    "license": "ISC",
    "dependencies": {
        "express": "^4.19.2"
    }
}
```

Semantic Versioning

- To keep the JavaScript ecosystem healthy, reliable, and secure.
  - MAJOR version when you make incompatible API changes,
  - MINOR version when you add functionality in a backwards compatible manner, and
  - PATCH version when you make backwards compatible bug fixes.



Semantic Versioning

- Dependencies are specified in a simple object that maps a package name to a version range. The version range is a string which has one or more space-separated descriptors.
    - **version :** Must match **version** exactly
    - **>version :** Must be greater than **version**
    - **~version :** “Approximately equivalent to version”
    - **^version:** “Compatible with version”

Example

- Suppose that we developed the project with 3.20.2 version of expressjs
- New versions:
  - 4.17.1
  - 4.17.0
  - 4.16.4
  - 4.16.3
  - 3.21.2
  - 3.21.1
  - 3.21.0
  - 3.20.3
  - **3.20.2**

	package.json	After “npm i”

No symbol	"dependencies": {
	"express": "3.20.2"
	}
		3.20.2 is installed

^	"dependencies": {
	"express": "^3.20.2"
	}
		The highest version with the same major number is installed **(3.21.2)**.

>	"dependencies": {
	"express": ">3.20.2"
	}
		> Using 3.20.2 ensures that the highest version is installed regardless of the major / minor that comes after 3.20.2. **(4.17.1)**

~	"dependencies": {
	"express": "~3.20.2"
	}
		Allows installing the highest version with the same minor version **(3.20.3)**

Package-lock.json

- The goal of the file is to keep track of the exact version of every package that is installed
- Product is 100% reproducible in the same way even if packages are updated by their maintainers.
- The `package-lock.json` sets your currently installed version of each package in stone, and `npm` will use those exact versions when running `npm install`.
- The `package-lock.json` file needs to be committed to your Git repository.

Any Question?

Thank you for your attention

esma.meral@ozyegin.edu.tr